

Bit Manipulation & Bitmasking Cheat Sheet (C-centric)

1. Bitwise Basics

Operation	C Code	Description
Set bit `i`	<code>`x  = (1 << i);`</code>	Sets bit `i` to 1
Clear bit `i`	<code>`x &= ~(1 << i);`</code>	Sets bit `i` to 0
Toggle bit `i`	<code>`x ^= (1 << i);`</code>	Flips bit `i`
Check bit `i`	<code>`x & (1 << i)`</code>	Is bit `i` set? (nonzero if yes)
Extract bit `i`	<code>`(x >> i) & 1`</code>	Returns 0 or 1

2. Counting & Manipulating Bits

Trick	C Code	Purpose
Count set bits (Kernighan)	<code>`while (x) { count++; x &= (x - 1); }`</code>	0(1s in x)
Clear lowest set bit	<code>`x & (x - 1)`</code>	Removes rightmost 1
Isolate lowest set bit	<code>`x & -x`</code>	Keeps only rightmost 1
Is x power of 2?	<code>`x > 0 && (x & (x - 1)) == 0`</code>	True only for powers of 2
XOR all nums to find single	<code>`res ^= x;`</code> in loop	Only unique survives

3. Bitmask Patterns

Pattern	C Code	Use Case
Subset loop over `n` bits	<code>`for (int mask = 0; mask < (1 << n); mask++)`</code>	Enumerate all subsets
Check if bit `i` is in mask	<code>`if (mask & (1 << i))`</code>	Include element `i` in subset
Count bits in a mask	Use Kernighan's or <code>`__builtin_popcount(mask)`</code> (GCC)	Count elements in subset

4. XOR Logic Patterns

Pattern	Result
<code>`x ^ x`</code>	0
<code>`x ^ 0`</code>	x
<code>`a ^ b ^ b`</code>	a
Used to swap:	<code>`a ^= b; b ^= a; a ^= b;`</code>

5. Advanced Tricks

Trick	C Code	Description
Reverse bits	Use loop: shift + mask	For FFT, binary mirror
Generate Gray Code	<code>`gray = n ^ (n >> 1)`</code>	Used in hardware encoders
Convert Gray -> Binary	<code>`while (shift) binary ^= (gray >> shift--)`</code>	Recursive XOR fold
Bit parity (even/odd 1s)	XOR all bits or use <code>`__builtin_parity()`</code>	HW parity check
Round up to power of 2	<code>`x--; x  = x >> 1; x  = x >> 2; ... x++;`</code>	Fills with 1s then adds 1